

Exercício 1: Questão Conceitual

Pergunta para a Turma:

Um aluno tentou executar o código PHP abaixo e recebeu um erro fatal do PHP no servidor. Observe atentamente:

```
class Usuario {  
    public $nome = "Ana";  
  
    public static function exibirNome() {  
        echo $this->nome; // ERRO FATAL!  
    }  
}
```

```
Usuario::exibirNome();
```

A) Qual é o erro cometido no código acima?

O erro é tentar usar o `$this->nome` dentro de um método estático (`static`). Isso gera o erro fatal: Fatal error: Uncaught Error: Using `$this` when not in object context.

B) Por que a variável pseudo-variável `$this` não funciona dentro de métodos declarado

Porque a variável `$this` só existe quando você cria um objeto de verdade na memória usando o `new` (ex: `$user = new Usuario()`). Métodos estáticos pertencem à classe, e não a um objeto instanciado. Como dá para chamar a função direto pela classe (`Usuario::exibirNome()`), o PHP roda esse código sem ter nenhum objeto criado por trás. Sem objeto, o `$this` não aponta para nada e o código quebra.

Exercício 2: Análise de Código

Analise o trecho de código abaixo e responda:

```
class Conversor {  
  
    public static function dolarParaReal($valor) {  
  
        return $valor * 5.00;  
  
    }  
  
}
```

// Opção A

```
$v1 = new Conversor();  
  
echo $v1->dolarParaReal(10);
```

// Opção B

```
echo Conversor::dolarParaReal(10);
```

Questão: Ambas as opções funcionam no PHP, mas qual delas representa a boa prática correta de orientação a objetos ao trabalhar com métodos estáticos? Justifique sua resposta.

A **Opção B** (`Conversor::dolarParaReal(10)`) é a forma conceitualmente correta porque métodos estáticos pertencem à classe, e não a um objeto específico. Quando um método é declarado como `static`, a intenção é justamente permitir sua chamada direta sem a necessidade de gastar memória instanciando um objeto (`new Conversor()`).

Exercício 3: Prática de Programação

Desafio de Código:

Escreva em PHP uma classe chamada `AcessoSistema` com os seguintes requisitos:

1. Crie uma propriedade estática privada chamada `$totalAcessos` iniciando em 0.
2. No método construtor `__construct()`, incremente o valor de `$totalAcessos` em +1.
3. Crie um método estático público `getAcessos()` que retorne a quantidade de acessos.
4. Instancie 4 objetos dessa classe e depois exiba na tela o resultado chamando `AcessoSistema::getAcessos()`.

```
<?php
```

```
class AcessoSistema {
```

```
    private static $totalAcessos = 0;
```

```
public function __construct() {  
  
    self::$totalAcessos++;  
  
}  
  
public static function getAcessos() {  
  
    return self::$totalAcessos;  
  
}  
  
}  
  
$a1 = new AcessoSistema();  
  
$a2 = new AcessoSistema();  
  
$a3 = new AcessoSistema();  
  
$a4 = new AcessoSistema();  
  
echo "Total de acessos: " . AcessoSistema::getAcessos(); // Saída: Total de  
acessos: 4  
  
?>
```